

Vous avez suivi **3 jours de cours intensifs** sur le volet NLP, en tandem avec la pipeline de HTR que vous avez développé en Computer Vision.

Vous présentez votre avancement **vendredi** (même si tout n'est pas finalisé), et vous disposez de **2 semaines supplémentaires** pour finaliser le projet après la soutenance.

Voici les consignes générales à suivre **spécifiquement pour la partie NLP** de votre projet. Elles vous aident à prioriser vos efforts, à éviter les pièges courants et à produire un livrable cohérent, même dans un temps réduit.

1. Ingestion du data contract – Ne passez pas à côté

Le JSON produit par votre pipeline HTR est le point de départ obligatoire de tout le NLP.

- **Validez systématiquement** le schéma avec `jsonschema` avant toute manipulation. Un champ manquant (`char_confidences` , `polygon`) vous bloquera plus tard.
- **Réalisez une EDA** (analyse exploratoire) même courte : calculez la distribution des confiances, le taux de `needs_review` , la longueur moyenne des lignes et le nombre d'abréviations résiduelles (ex. `~` , `p` , `q`). Ces chiffres vous permettent de justifier vos choix (ex. seuil de confiance pour la correction).
- **Sceliez votre split train/val/test dès le départ** (stratifié par siècle et type de document). Calculez le SHA-256 du test set et ne le regardez plus jusqu'à la fin.

2. Normalisation – Priorité absolue (règles d'abord, IA ensuite)

La normalisation est la brique la plus facile à mettre en œuvre et celle qui apporte le plus de gain en CER immédiat.

- **Commencez par les règles déterministes** : Unicode NFC, substitution u/v et i/j, résolution du tilde nasal, table d'abréviations (ex. `q~` → `que`). Documentez chaque règle dans `CONVENTIONS_NLP.md` .
- **Adaptez les règles à votre corpus** : ne recopiez pas simplement les règles génériques du cours. Par exemple, si votre corpus contient beaucoup de formes latines, ajustez la table d'abréviations en conséquence.
- **Implémentez la correction guidée par confiance** : utilisez les `char_confidences` et `candidates` fournis par le HTR.
 - **Le principe** : pour chaque position où la confiance est faible (ex. `< 0.7`) et où plusieurs candidats sont proposés (`"options": ["a", "e", "o"]`), on utilise un

modèle de langue pour choisir le candidat le plus probable dans le contexte.

- OU utilisez les `char_confidences` et le lexique pour trouver les mots les plus probables.
- **Le modèle : CamemBERT** (`almanach/camembert-base`) en mode **MLM** (Masked Language Model). On masque le mot contenant la position ambiguë, on évalue chaque candidat, et on choisit celui qui maximise la probabilité du token masqué.
- **Modèles pré-entraînés sur données médiévales** : pour améliorer la pertinence du modèle face aux graphies anciennes, vous pouvez utiliser des modèles **CamemBERT déjà fine-tunés sur des corpus médiévaux** disponibles sur Hugging Face. Recherchez des modèles comme `magisttermilitum/roberta-multilingual-medieval-ner` (pour du vieux français) ou `pjox/dalembert-classical-fr-ner` pour du moyen français ou des modèles issus de projets comme **CREMMA** ou **CATMuS** (consultez Hugging Face avec les mots-clés `camembert medieval` ou `camembert old french`). Ces modèles sont plus robustes aux variantes orthographiques du moyen français.
- **Mesurez le CER avant/après chaque étape** (règles seules, puis correction guidée).
 - **Problème** : vous travaillez sur des manuscrits sans vérité terrain. Impossible de calculer un CER absolu (comparaison à une référence humaine).
 - **Solution** : utilisez une **évaluation relative** en comparant différentes versions d'une **même transcription** entre elles. L'outil [Evaluation-HTR](#) a été conçu pour cela : il permet de comparer plusieurs itérations d'une même transcription (par exemple, la sortie brute du HTR, une version pour chaque règle de normalisation, plusieurs itérations de la normalisation intégrant les correction du lexique, etc) et de calculer les **distances relatives** entre elles. Cela vous donne une courbe d'évolution sans avoir besoin de vérité terrain. Vous pouvez ainsi mesurer objectivement l'apport de chaque étape de votre pipeline. Cet outil est en cours de développement mais vous pouvez vous baser sur son fonctionnement pour mesurer votre CER.

Dans les faits vous pouvez-vous arrêter là, ceux qui veulent peuvent essayer les étapes suivantes.

Evidemment vous ne présenterez Vendredi que les étapes réalisées ou que vous souhaitez réaliser

3. NER – Partez d'un modèle existant, fine-tunez légèrement

La NER est la tâche NLP la plus exigeante en termes de données annotées. Avec peu de temps, **ne partez pas de zéro** : utilisez un modèle CamemBERT déjà fine-tuné sur des

données proches (français général, données littéraires, voire médiévales) et **adaptez-le** à votre corpus par un fine-tuning léger.

- **Choisissez un modèle de base adapté** : plusieurs modèles CamemBERT NER sont disponibles sur Hugging Face et peuvent servir de point de départ, comme ceux mentionnés plus haut.
- **Adaptez le schéma d'annotation** : Si vous utilisez un modèle existant, vérifiez ses classes (`PER` , `LOC` , `ORG` , `MISC` , `DATE`). Ajoutez éventuellement une classe `TITLE` si vos textes contiennent beaucoup de titres (roi, comte, évêque). Vous devrez alors **étendre le classifieur** du modèle (modifier `num_labels`) et le fine-tuner sur vos données.
- **Validez manuellement un petit échantillon** (200-300 tokens) pour avoir un jeu d'entraînement minimal. Même si le F1 final est faible (0,40-0,50), cela vous permet de valider l'intégralité du pipeline.
- **L'alignement des labels sur la tokenisation est le point critique** : assurez-vous de comprendre l'utilisation de `-100` pour les sous-tokens de continuation (word-pieces). Montrez cette étape dans votre code et votre présentation.
- **Évaluez avec seqeval** : F1 micro et F1 par type d'entité. Si les chiffres sont bas, analysez qualitativement les erreurs (ex. confusion entre `TITLE` et `PER`) – c'est aussi une forme de résultat scientifique.

4. POS, graphe et TEI

Ces étapes sont moins chronophages et peuvent être réalisées sur un sous-ensemble du corpus (quelques pages).

- Pour le **POS** et les **lemmes**, utilisez `stanza` avec le modèle `frm` (moyen français) ou `pie-extended medieval-fr`. L'exécution est rapide (quelques secondes par page).
- Pour l'**extraction de relations**, **ne cherchez pas à faire du prompting LLM complexe** avec le peu de temps disponible. Implémentez des **règles lexico-syntaxiques simples** (ex. regex sur les séquences de labels NER : `B-PER .*` `prist .*` `B-LOC` → relation (`PERSONNE` , `PREND` , `LIEU`)). Cela suffit pour prouver que vous avez compris le concept.
- Pour le **graphe** (`NetworkX`) et l'**export TEI-XML**, ciblez un petit échantillon (5 à 10 pages). Balisez les entités avec `<persName>` , `<placeName>` , `<date>` dans un fichier TEI valide. Un extrait bien présenté vendredi vaut mieux qu'un export complet mais bâclé.

5. Code, tests et documentation – Tenez-vous aux standards

Même avec un pipeline imparfait, une documentation de qualité et un code structuré sont très valorisés.

- **Structurez votre dépôt** : séparez clairement les modules CV (`src/htr/`) et NLP (`src/nlp/`).
- **Documentez vos choix NLP** dans un fichier `CONVENTIONS_NLP.md` (pourquoi cette normalisation ? pourquoi ce schéma BIO ?).
- **Écrivez au moins 2 tests `pytest`** : un pour valider le schéma JSON du data contract, un pour vérifier que la normalisation par règles ne dégrade pas le CER sur un petit échantillon de référence. Cela montre une rigueur d'ingénierie appréciée.
- **Fixez les seeds** et versionnez vos dépendances (`pyproject.toml`). La reproductibilité est un critère d'évaluation à part entière.